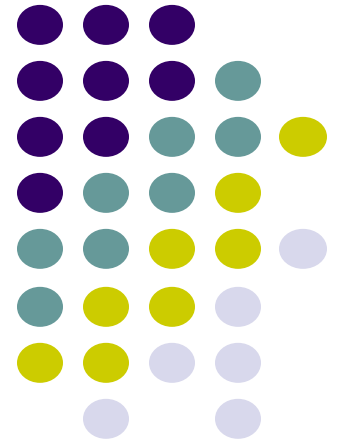


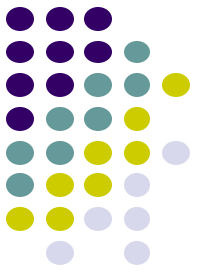
ErSE222: Machine learning in Geoscience

April. 6th, 2025

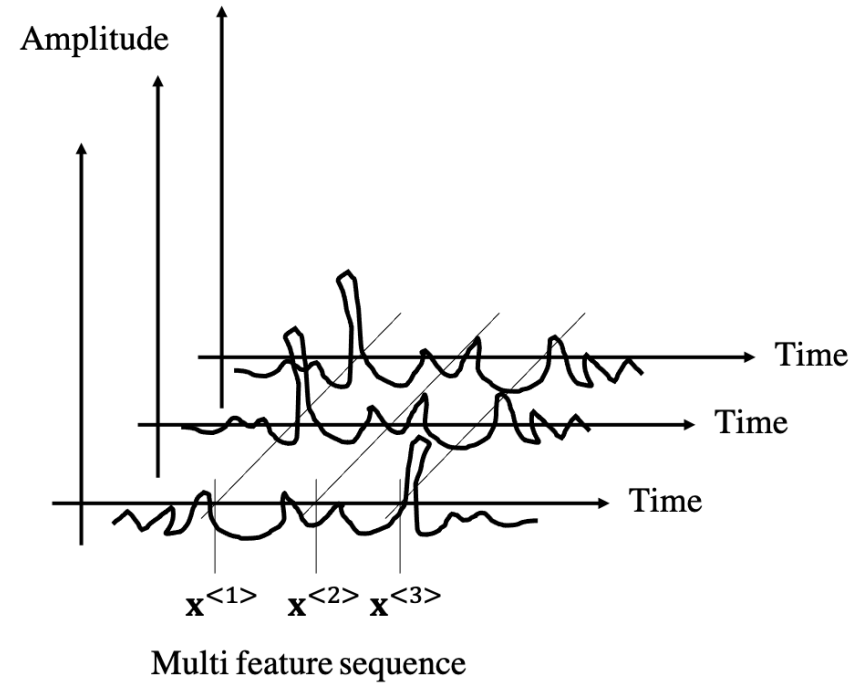
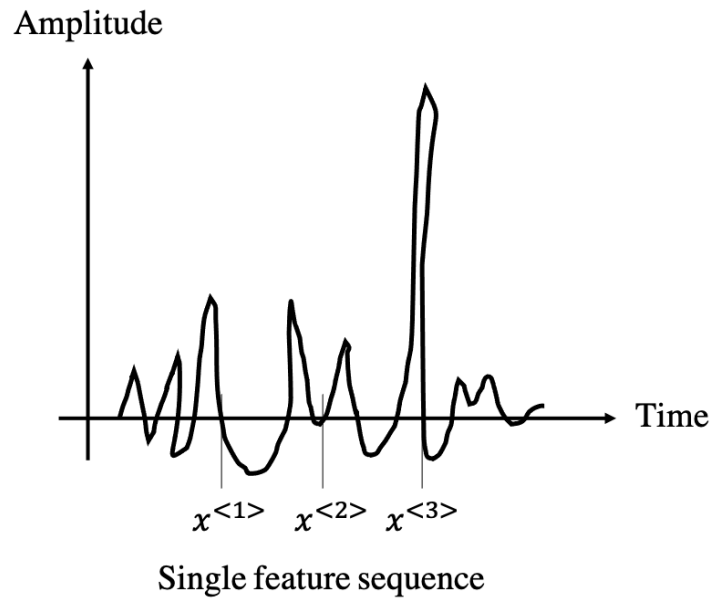
Tariq Alkhalifah and Omar Saad



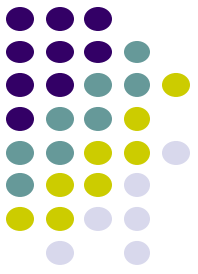
Sequence modelling



Learning tasks that involve sequences as input data



Sequence modelling applications



1. Single-output classification (x sequence, y single value):

given an input sequence of a certain length, a model is trained to decide whether the sequence contains a feature of interest or not

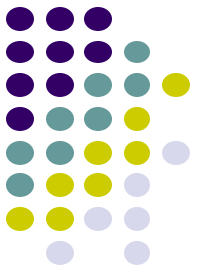
2. Multi-output classification / semantic segmentation (x and y sequences):

given an input sequence of a certain length x , a model is trained to classify each element of the input sequence into a predefined set of classes.

3. Regression (x sequence, y single value or sequence):

given an input sequence of a certain length, a model is trained to predict a continuous value or a sequence of continuous values

Sequence modelling with known tools



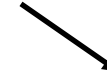
Sequence with N_τ 'time' samples and N_f features

$$\mathbf{X} = \begin{bmatrix} x_1^{<1>} & x_1^{<2>} & x_1^{N_\tau} \\ \dots & \dots & \dots \\ x_{N_f}^1 & x_1^{<2>} & x_{N_f}^{N_\tau} \end{bmatrix} = [\mathbf{x}^{<1>} \quad \mathbf{x}^{<2>} \quad \mathbf{x}^{<N_\tau>}]_{[N_f \times N_\tau]}$$



2D array as-is: CNNs

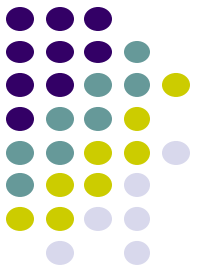
*Locality of conv filters does not
make much sense*



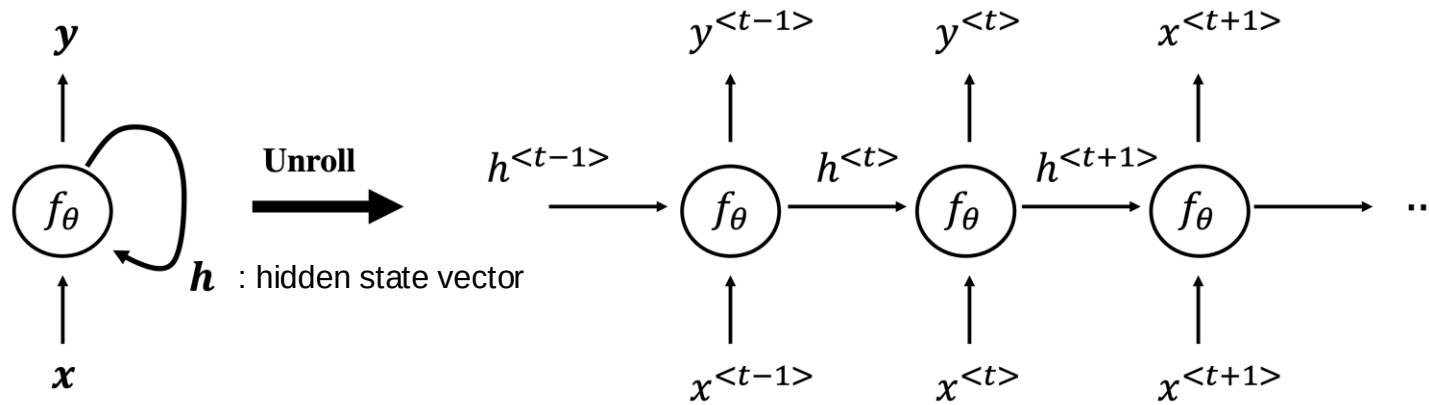
1D array ($vec(\mathbf{X})$): MLP

- $vec(\mathbf{X})$ can be very large and so the weight matrices
- cannot handle sequences of variable length

Sequence modelling with new tools



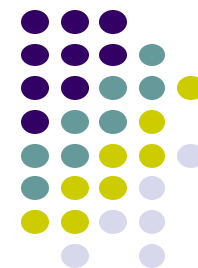
Solution: parameter sharing via Recurrent Neural Networks (RNNs)



RNN

Graph-Unrolled RNN

Useful notation



$$\mathbf{X} = [\mathbf{x}^{<1>} \quad \mathbf{x}^{<2>} \quad \mathbf{x}^{<T_x>}]_{[N_f \times T_x]} \quad \mathbf{Y} = [\mathbf{y}^{<1>} \quad \mathbf{y}^{<2>} \quad \mathbf{y}^{<T_y>}]_{[N_t \times T_y]}$$

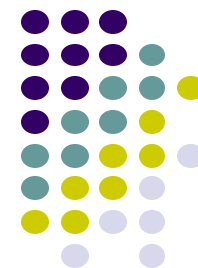
T_x : length of input sequence (can be sample dependent, so we will use $T_x^{(i)}$)

T_y : length of output sequence (can be sample dependent, so we will use $T_y^{(i)}$)

N_f : number of features of input

N_t : number of elements of output

Useful notation



$$\mathbf{X} = [\mathbf{x}^{<1>} \quad \mathbf{x}^{<2>} \quad \mathbf{x}^{<T_x>}]_{[N_f \times T_x]} \quad \mathbf{Y} = [\mathbf{y}^{<1>} \quad \mathbf{y}^{<2>} \quad \mathbf{y}^{<T_y>}]_{[N_t \times T_y]}$$

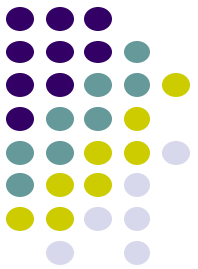
- A single training sample has two dimensions

- An entire training data has three dimensions:

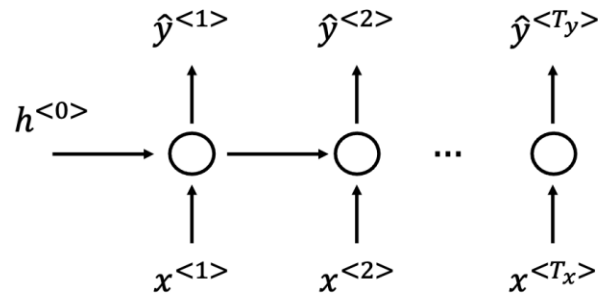
$$\text{Input} \quad [N_s \times N_f \times T_x]$$

$$\text{Target} \quad [N_s \times N_t \times T_y]$$

Common sequence models



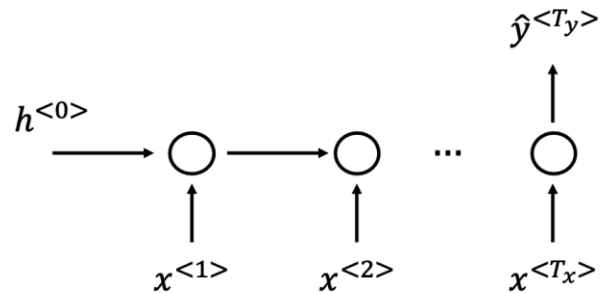
1. $T_x = T_y$



Many-to-many architecture

Segmentation or domain translation

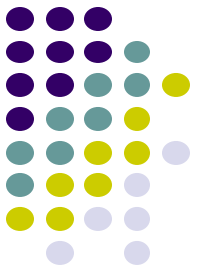
2. $T_x, T_y = 1$



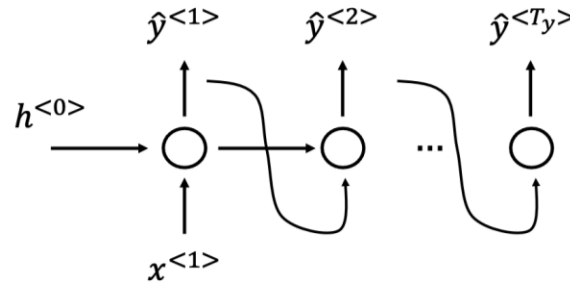
Many-to-one architecture

Classification or regression

Common sequence models

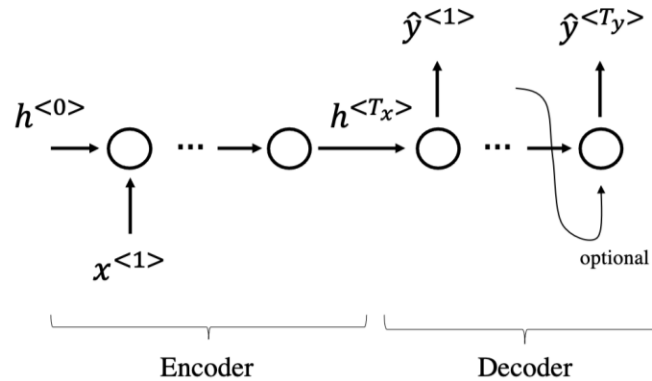


3. $T_x = 1, T_y$



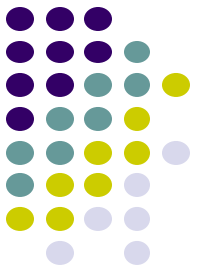
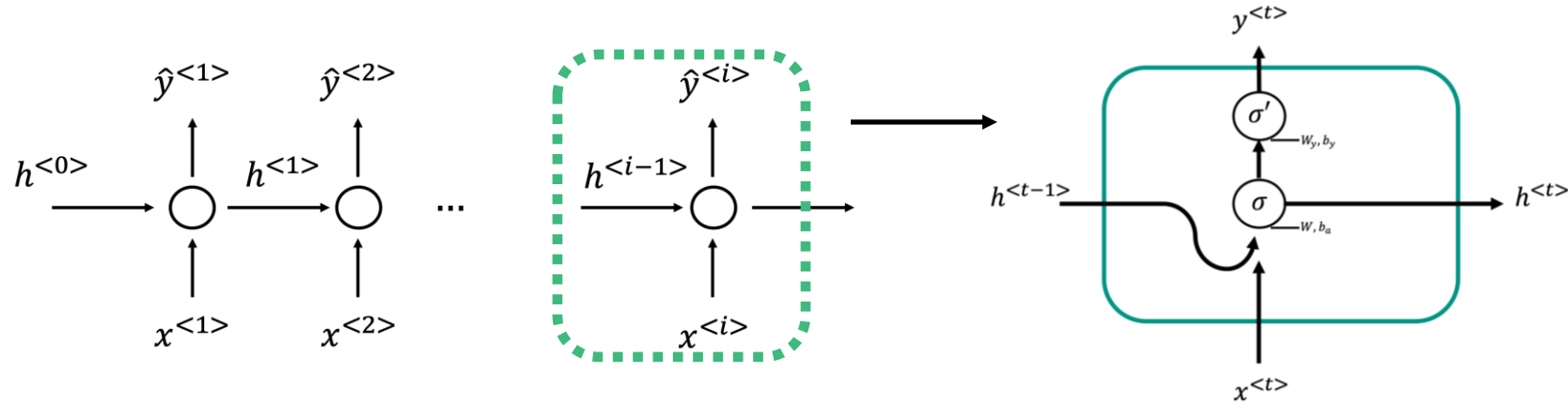
One-to-many architecture
(for generative tasks)

4. $T_x \neq T_y$



Encoder-decoder or
sequence-to-sequence
architecture

Basic RNN architecture



$$\mathbf{a}^{<t>} = \mathbf{W}_h \mathbf{h}^{<t-1>} + \mathbf{W}_x \mathbf{x}^{<t>} + \mathbf{b}_a = \mathbf{W} [\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}]^T + \mathbf{b}_a$$

$$\mathbf{h}^{<t>} = \sigma(\mathbf{a}^{<t>})$$

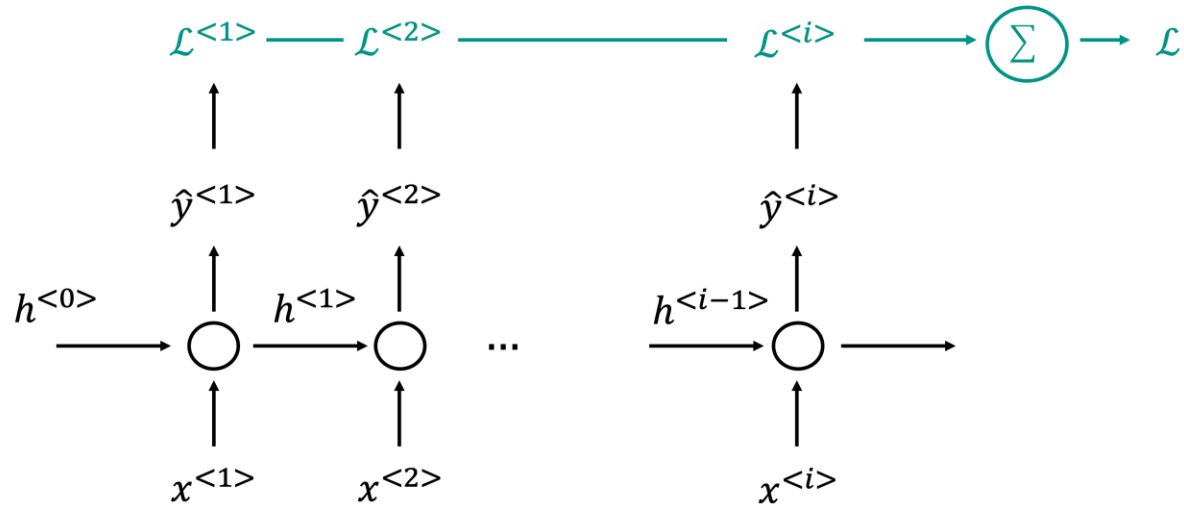
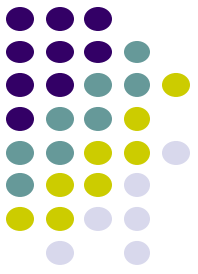
$$\mathbf{o}^{<t>} = \mathbf{W}_y \mathbf{h}^{<t>} + \mathbf{b}_y$$

$$\hat{\mathbf{y}}^{<t>} = \sigma'(\mathbf{o}^{<t>})$$

$$\mathbf{W} = [\mathbf{W}_h, \mathbf{W}_x]_{[N_h \times N_h + N_x]}$$

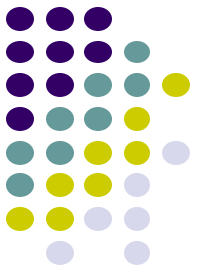
$$\mathbf{W}_y \text{ } [N_y \times N_h]$$

Basic RNN loss function

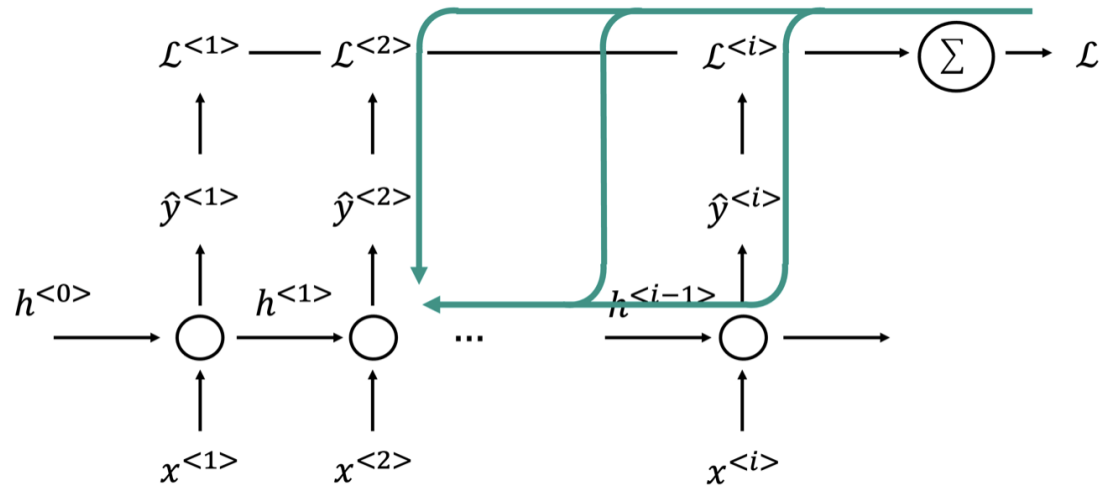


$$\mathcal{L} = \sum_{t=1}^{T_x} \mathcal{L}^{<t>}, \quad \mathcal{L}^{<t>} = f(\hat{\mathbf{y}}^{<t>}, \mathbf{y}^{<t>})$$

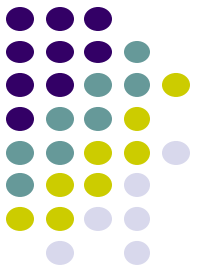
Basic RNN back-propagation



Applied on the unrolled graph $\rightarrow$ back-propagation through time (BPTT)

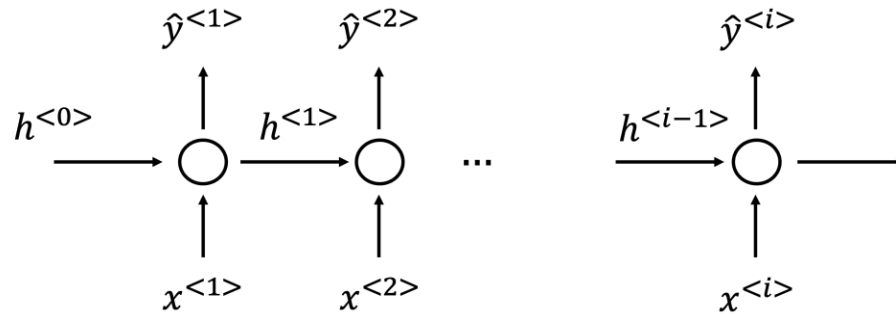


Bidirectional RNNs

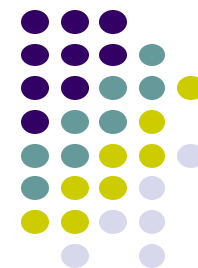


Unidirectional RNNs learn in a causal fashion $\rightarrow$ online processing

$$x^{<t-T_x>}, \dots, x^{<t-2>} x^{<t-1>} \longrightarrow y^{<t>}$$



Bidirectional RNNs



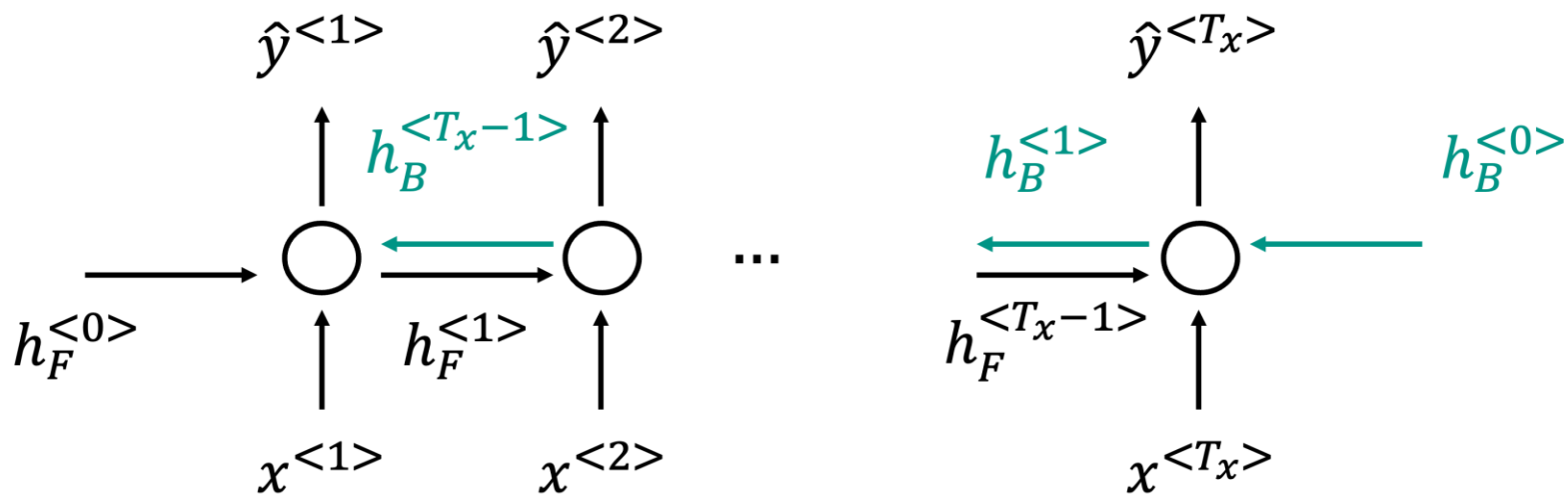
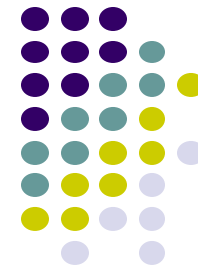
Unidirectional RNNs learn in a causal fashion → online processing

$$x^{<t-T_x>}, \dots, x^{<t-2>} x^{<t-1>} \longrightarrow y^{<t>}$$

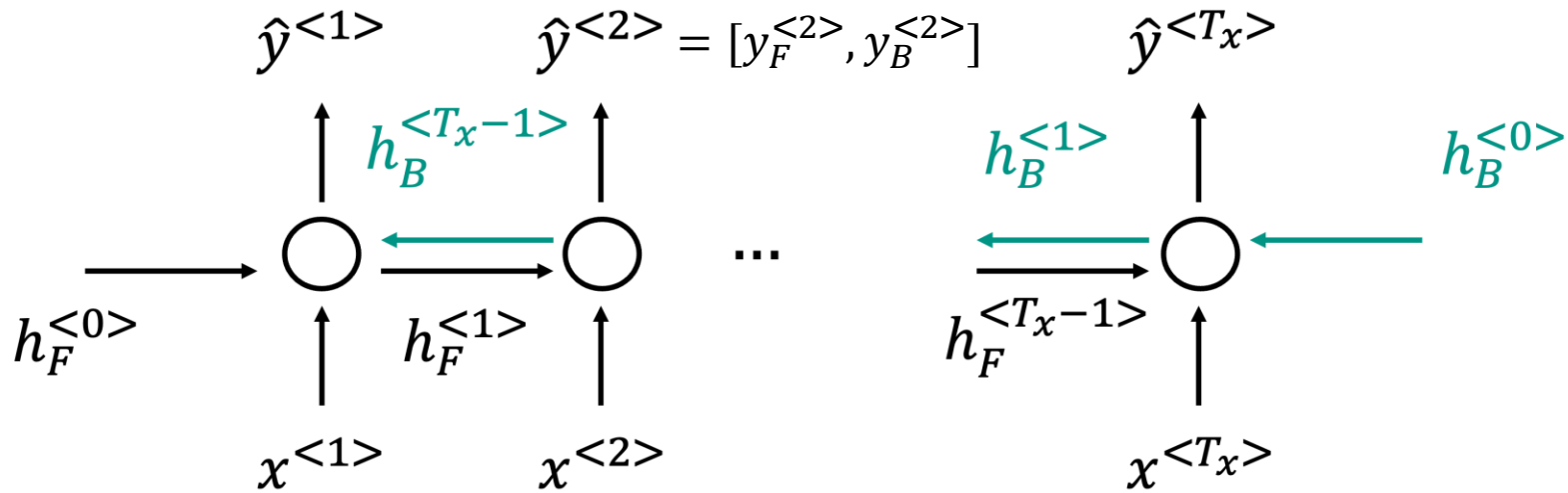
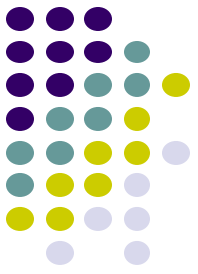
Bidirectional RNNs can learn from past & future → offline processing

$$x^{<t-T_x>}, \dots, x^{<t-1>}, x^{<t>}, x^{<t+1>}, \dots, x^{<t+T_x>} \longrightarrow y^{<t>}$$

Bidirectional RNNs



Bidirectional RNNs



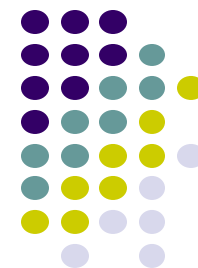
Causal direction

($h_F^{<\cdot>}$: hidden states of network moving forward)

Acausal direction

($h_B^{<\cdot>}$: hidden states of network moving backward)

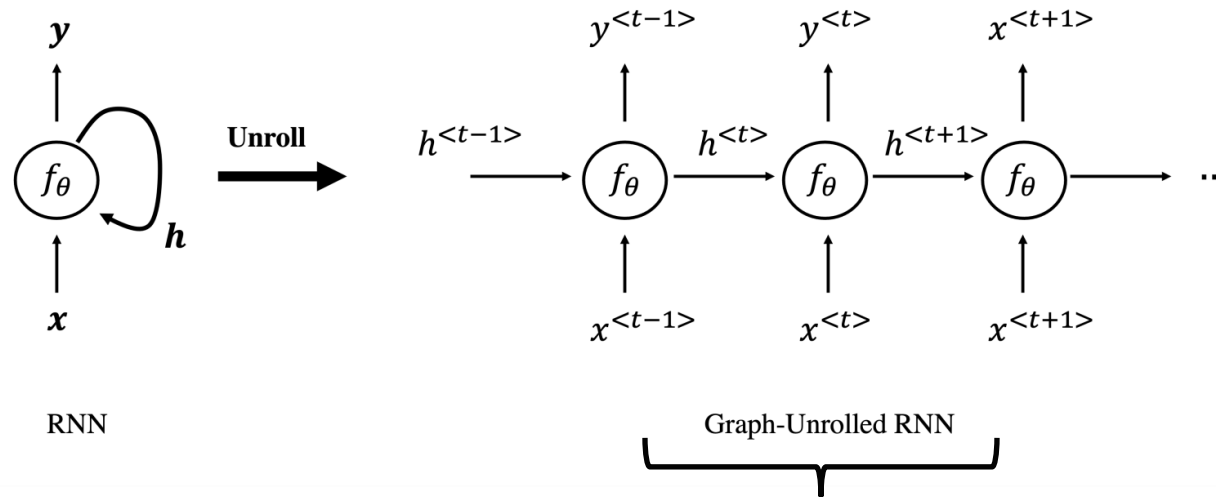
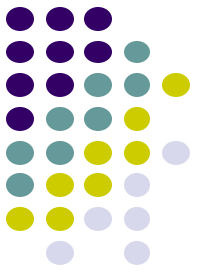
Deep RNNs



Like any other NN, RNNs can be stacked together:

- Shallow RNN: single hidden layer between input and output
- Deep RNN: multiple hidden layers between input and output
 - **First layer** input: $\mathbf{x}^{<t>}$, hidden and output: $\mathbf{h}_0^{<t>}$,
 - **Second layer** input: $\mathbf{h}_0^{<t>}$, hidden and output: $\mathbf{h}_1^{<t>}$,
 - **Last layer** input: $\mathbf{h}_{N-1}^{<t>}$, hidden: $\mathbf{h}_N^{<t>}$, output: $\hat{\mathbf{y}}^{<t>}$.

Long-term dependencies

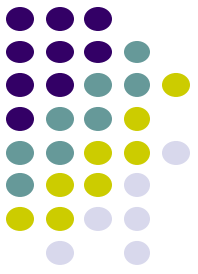


The same matrix $\mathbf{W}_h$ is repeatedly applied to the hidden state vectors

vanishing & exploding gradients

Long-term dependencies become insignificant compared to short-term ones

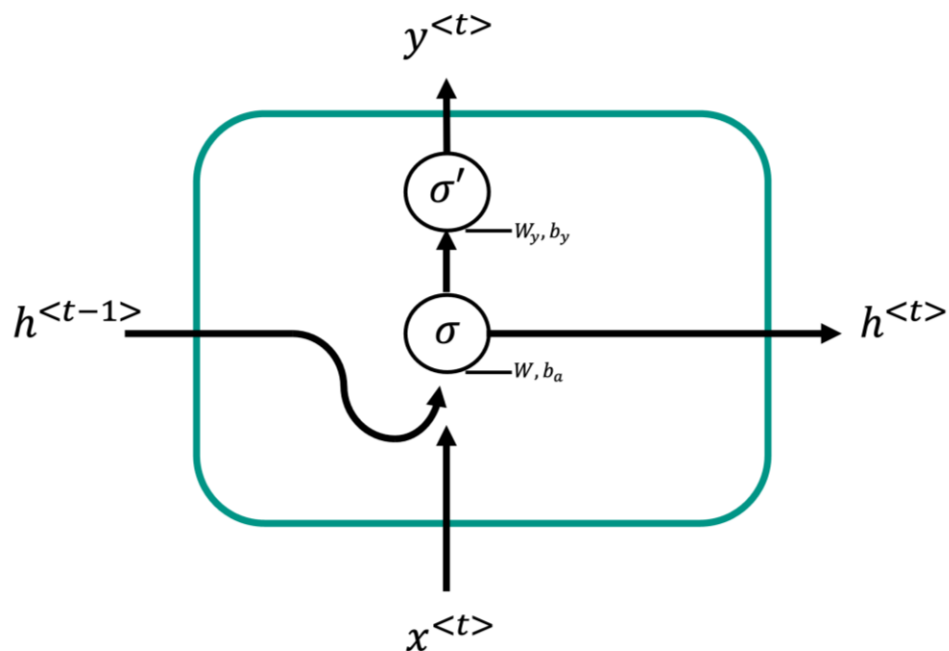
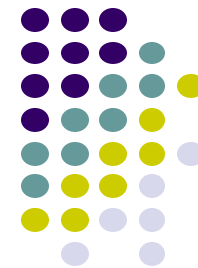
Long-term dependencies



Solutions

- Tackled in training with gradient clipping
- Modified architecture
 - Gated Recurrent Unit (GRU)
 - Long-short term memory Unit (LSTM)

Basic RNN



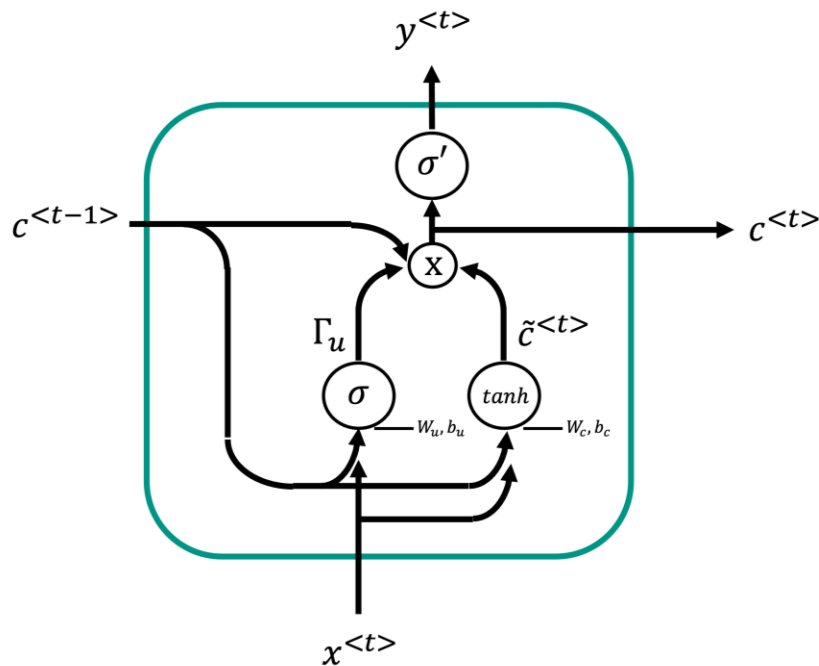
$$\mathbf{a}^{<t>} = \mathbf{W}_h \mathbf{h}^{<t-1>} + \mathbf{W}_x \mathbf{x}^{<t>} + \mathbf{b}_a$$

$$\mathbf{h}^{<t>} = \sigma(\mathbf{a}^{<t>})$$

$$\mathbf{o}^{<t>} = \mathbf{W}_y \mathbf{h}^{<t>} + \mathbf{b}_y$$

$$\hat{\mathbf{y}}^{<t>} = \sigma'(\mathbf{o}^{<t>})$$

Simplified GRU



$$\Gamma_u = \sigma \left(\mathbf{W}_u \begin{bmatrix} \mathbf{c}^{<t-1>} \\ \mathbf{x}^{<t>} \end{bmatrix} + \mathbf{b}_u \right)$$

$$\tilde{\mathbf{c}}^{<t>} = \tanh \left(\mathbf{W}_c \begin{bmatrix} \mathbf{c}^{<t-1>} \\ \mathbf{x}^{<t>} \end{bmatrix} + \mathbf{b}_c \right)$$

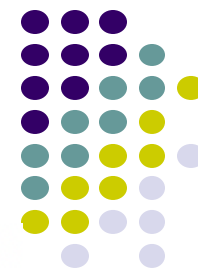
$$\mathbf{c}^{<t>} = \Gamma_u \cdot \tilde{\mathbf{c}}^{<t>} + (1 - \Gamma_u) \cdot \mathbf{c}^{<t-1>}$$

$$\hat{\mathbf{y}}^{<t>} = \sigma'(\mathbf{W}_y \mathbf{c}^{<t>} + \mathbf{b}_y)$$

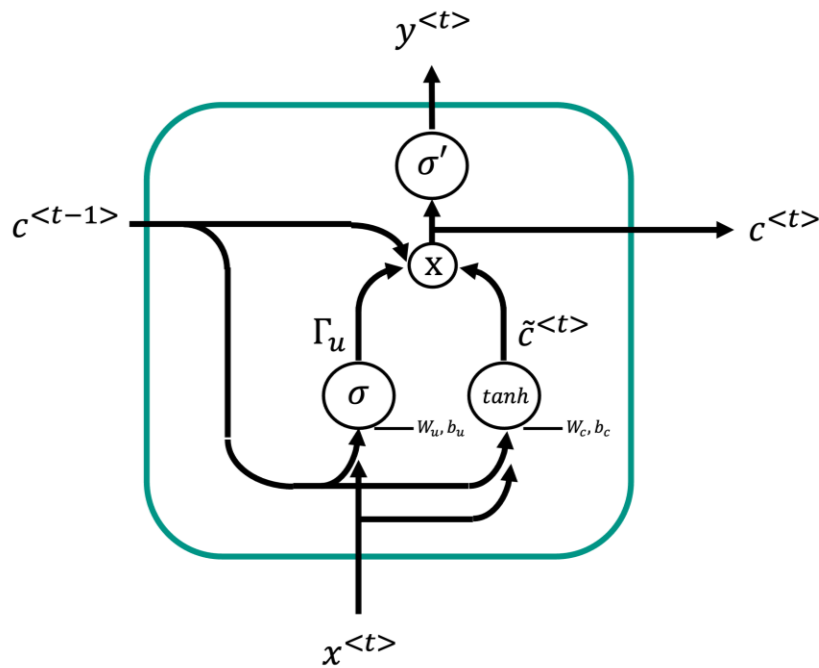
$\mathbf{c}^{<t>}$: memory cell (same as hidden state vector)

$\tilde{\mathbf{c}}^{<t>}$: candidate replacement for memory cell

Γ_u : update gate



Simplified GRU



Sigmoid acting as soft-switch: 0 keep old, 1 update

$$\Gamma_u = \sigma \left(\mathbf{W}_u \begin{bmatrix} \mathbf{c}^{<t-1>} \\ \mathbf{x}^{<t>} \end{bmatrix} + \mathbf{b}_u \right)$$

$$\tilde{\mathbf{c}}^{<t>} = \tanh \left(\mathbf{W}_c \begin{bmatrix} \mathbf{c}^{<t-1>} \\ \mathbf{x}^{<t>} \end{bmatrix} + \mathbf{b}_c \right)$$

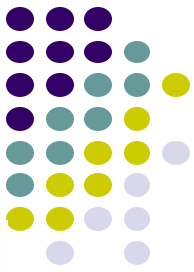
$$\mathbf{c}^{<t>} = \Gamma_u \cdot \tilde{\mathbf{c}}^{<t>} + (1 - \Gamma_u) \cdot \mathbf{c}^{<t-1>}$$

$$\hat{\mathbf{y}}^{<t>} = \sigma'(\mathbf{W}_y \mathbf{c}^{<t>} + \mathbf{b}_y)$$

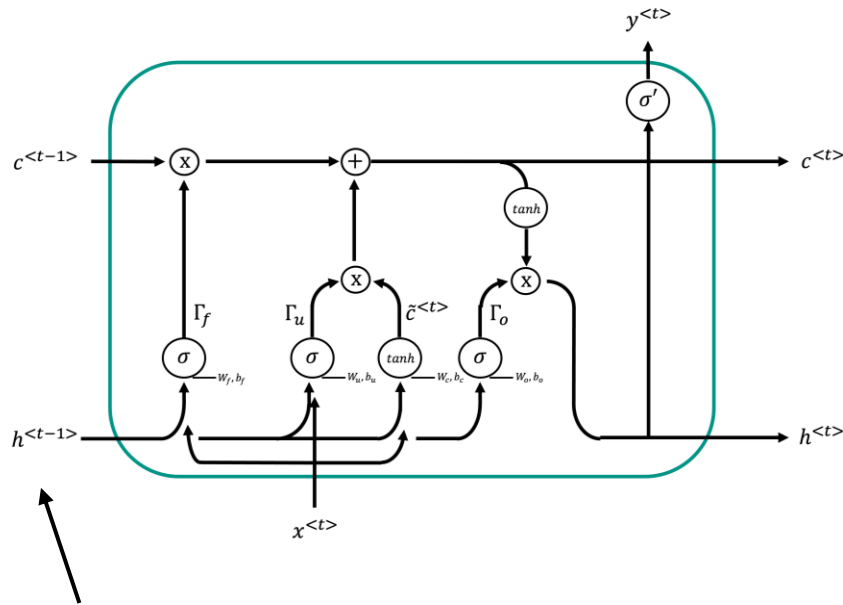
$\mathbf{c}^{<t>}$: memory cell (same as hidden state vector)

$\tilde{\mathbf{c}}^{<t>}$: candidate replacement for memory cell

Γ_u : update gate $\rightarrow$ can stop info flowing for a number of step creating long term dependencies



LSTM



Two hidden states

$$\Gamma_u = \sigma \left(W_u \begin{bmatrix} h^{<t-1>} \\ x^{<t>} \end{bmatrix} + b_u \right)$$

$$\Gamma_f = \sigma \left(W_f \begin{bmatrix} h^{<t-1>} \\ x^{<t>} \end{bmatrix} + b_f \right)$$

$$\Gamma_o = \sigma \left(W_o \begin{bmatrix} h^{<t-1>} \\ x^{<t>} \end{bmatrix} + b_o \right)$$

$$\tilde{c}^{<t>} = \tanh \left(W_c \begin{bmatrix} h^{<t-1>} \\ x^{<t>} \end{bmatrix} + b_c \right)$$

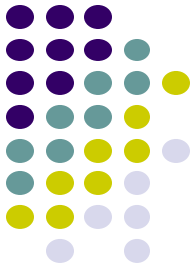
$$c^{<t>} = \Gamma_u \tilde{c}^{<t>} + \Gamma_f c^{<t-1>}$$

$$h^{<t>} = \Gamma_o \tanh(c^{<t>})$$

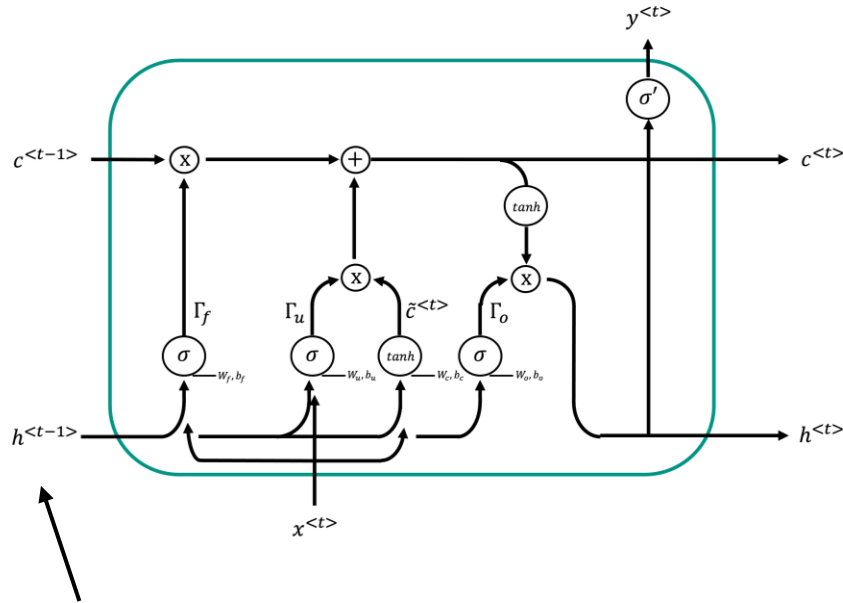
$$y^{<t>} = \sigma' (W_y h^{<t>} + b_y)$$

Γ_f : forget gate

Γ_o : output gate



LSTM



Two hidden states

$$\Gamma_u = \sigma \left(W_u \begin{bmatrix} h^{<t-1>} \\ x^{<t>} \end{bmatrix} + b_u \right)$$

$$\Gamma_f = \sigma \left(W_f \begin{bmatrix} h^{<t-1>} \\ x^{<t>} \end{bmatrix} + b_f \right)$$

$$\Gamma_o = \sigma \left(W_o \begin{bmatrix} h^{<t-1>} \\ x^{<t>} \end{bmatrix} + b_o \right)$$

$$\tilde{c}^{<t>} = \tanh \left(W_c \begin{bmatrix} h^{<t-1>} \\ x^{<t>} \end{bmatrix} + b_c \right)$$

$$c^{<t>} = \Gamma_u \tilde{c}^{<t>} + \Gamma_f c^{<t-1>}$$

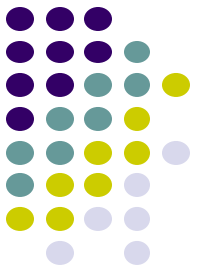
$$h^{<t>} = \Gamma_o \tanh(c^{<t>})$$

$$y^{<t>} = \sigma'(W_y h^{<t>} + b_y)$$

Γ_f : forget gate → compared to GRU, this is learned and not $1 - \Gamma_u$

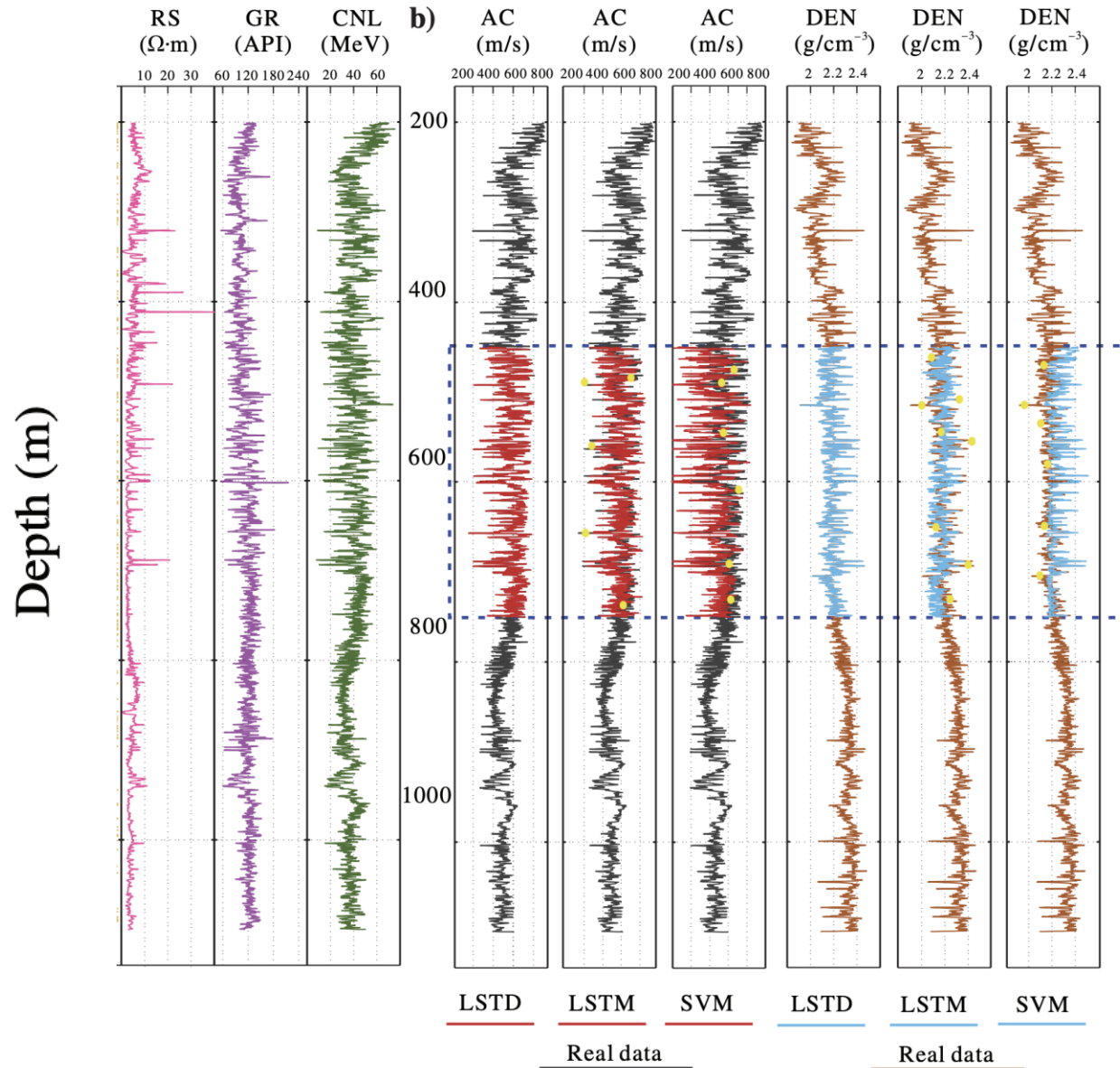
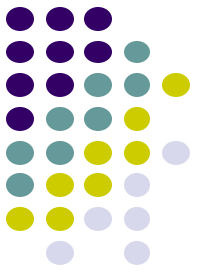
Γ_o : output gate



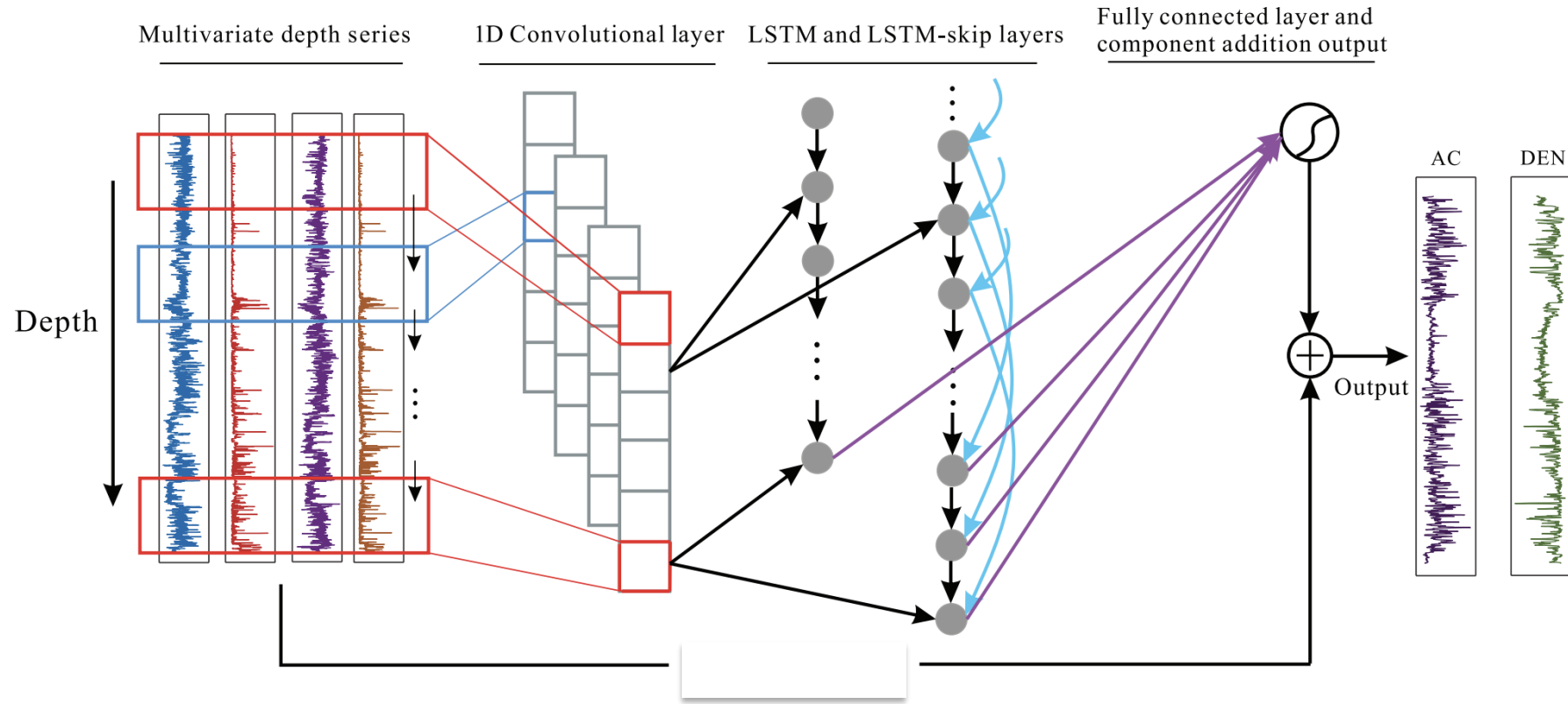


Application

Missing well-log prediction



Missing well-log prediction



Deep-learning missing well-log prediction via long short-term memory network with attention-period mechanism